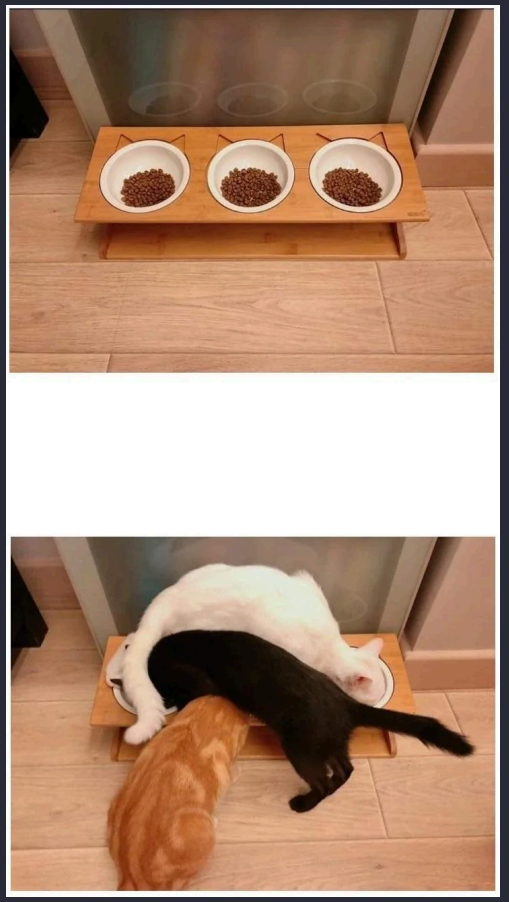
Error Handling

CS-3180

Errors

- Errors a *normal* part of programming
- Errors are *expected*
- Assumptions of correctness rarely lead to good software
- Can never cover every case



Dealing with Errors

- Different languages approach errors differently
- Some treat as a normal part of develops, others not so much
- Changed dramatically throughout the years
- Look at three approaches to this
- Dive into Rust's approach

C Error Handling

- Very basic system
- Checking NULL and return integer codes

```c
#include <stdio.h>
#include <errno.h>

int main() {
    // writes to a global errno variable
    FILE *fp = fopen("non_existent_file.txt", "r");

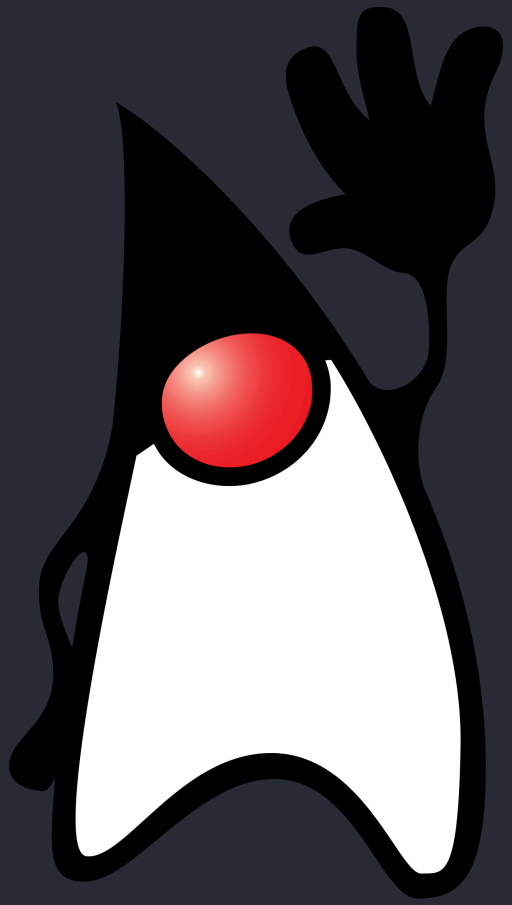
    if (fp == NULL) {
        printf("Error code: %d\n", errno);
    }
    return 0;
}
```

Error Codes

- Main's return becomes the exit code (checked later)
- This isn't really even an error system
- Just checking whether things are valid
- Good practice, but it fails to scale well to large software

Types

- A better type system can solve this
- Build types into the language
- Maybe throw in specific syntax
- This is how Java got its error handling
- But we'll pick on Python



Exceptions

- Many modern error handling is built around Exceptions
- Building types for *what* could go wrong in your code
- Introduce expression blocks for both cases
- `try` block for code that could fail
- `catch/except` block(s) for how to handle it
- `finally` block for cleanup (always runs)

Python Exceptions

- Specify different exceptions that *could* occur

```
try:
    file = open("data.txt", "r")
    value = 10 / int(file.readline())
except FileNotFoundError as e:
    print(f"File missing: {e}")
except ZeroDivisionError:
    print("Cannot divide by zero.")
finally:
    print("Cleaning up resources...")
    try:
        file.close()
    except NameError:
        pass
```

Python Exceptions

- Specify different exceptions that *could* occur

```
try:
    file = open("data.txt", "r")
    value = 10 / int(file.readline())
except FileNotFoundError as e:
    print(f"File missing: {e}")
except ZeroDivisionError:
    print("Cannot divide by zero.")
finally:
    print("Cleaning up resources...")
    try:
        file.close()
    except NameError:
        pass
```

Python Exceptions

- Specify different exceptions that *could* occur

```
try:
    file = open("data.txt", "r")
    value = 10 / int(file.readline())
except FileNotFoundError as e:
    print(f"File missing: {e}")
except ZeroDivisionError:
    print("Cannot divide by zero.")
finally:
    print("Cleaning up resources...")
    try:
        file.close()
    except NameError:
        pass
```

Python Exceptions

- Specify different exceptions that *could* occur

```
try:
    file = open("data.txt", "r")
    value = 10 / int(file.readline())
except FileNotFoundError as e:
    print(f"File missing: {e}")
except ZeroDivisionError:
    print("Cannot divide by zero.")
finally:
    print("Cleaning up resources...")
    try:
        file.close()
    except NameError:
        pass
```

Python Exceptions

- Specify different exceptions that *could* occur

```
try:
    file = open("data.txt", "r")
    value = 10 / int(file.readline())
except FileNotFoundError as e:
    print(f"File missing: {e}")
except ZeroDivisionError:
    print("Cannot divide by zero.")
finally:
    print("Cleaning up resources...")
    try:
        file.close()
    except NameError:
        pass
```

Python Exceptions

- Specify different exceptions that *could* occur

```
try:
    file = open("data.txt", "r")
    value = 10 / int(file.readline())
except FileNotFoundError as e:
    print(f"File missing: {e}")
except ZeroDivisionError:
    print("Cannot divide by zero.")
finally:
    print("Cleaning up resources...")
    try:
        file.close()
    except NameError:
        pass
```

Exceptions Systems

- Java goes even further and differentiates between
- Compile-time (*checked*) exceptions
- Run-time (*un-checked*) exceptions
- Also differentiates between recoverable and non-recoverable
- Same `try/catch/finally` style of error handling

Try/Catch Issues

- This is improvement on C for sure
- Developers tend to take issue with a few things
- Problem lies in the control flow
- Code is not read as it is executed
- Re-examine that example

Hidden Goto

- Try can jump to many places
- Could also raise a `ValueError`, that lacks a `except`
- Silently jump to the caller, a catch block, or a finally

```
try:
    file = open("data.txt", "r")
    value = 10 / int(file.readline())
except FileNotFoundError as e:
    print(f"File missing: {e}")
except ZeroDivisionError:
    print("Cannot divide by zero.")
finally:
    print("Cleaning up resources...")
    try:
        file.close()
    except NameError:
        pass # could be un-initialized
```

Where to go now?

- Fundamental problem is about interruption
- Standard flow is stopped, passed elsewhere
- Dealt with via special syntax
- Instead of treating these as "exceptions"
- Take a page out of C's book, treated them as values
- Returned both the data, and any errors

Errors as Values

- Golang says this is a simple problem to solve
- Every function that could fail, returns a tuple
- Normal function *return value*
- And an *Error value*
- Check it everytime you call it!



Go Errors

- Return type is `(int, error)`
- Return `nil` error if success
- Check both when function is called

```go
func divide(a, b int) (int, error) {
  if b == 0 {
      return 0, errors.New("Can't divide by 0")
  }

  return a / b, nil
}

func main() {
  result, err := divide(10, 2)
  if err != nil {
    fmt.Println("Error caught:", err)
    return
  }

  fmt.Println("Success, result was: ", result)
```

Go Errors

- Return type is `(int, error)`
- Return `nil` error if success
- Check both when function is called

```go
func divide(a, b int) (int, error) {
  if b == 0 {
      return 0, errors.New("Can't divide by 0")
  }

  return a / b, nil
}

func main() {
  result, err := divide(10, 2)
  if err != nil {
    fmt.Println("Error caught:", err)
    return
  }

  fmt.Println("Success, result was: ", result)
```

Go Errors

- Return type is `(int, error)`
- Return `nil` error if success
- Check both when function is called

```
2    if b == 0 {
3        return 0, errors.New("Can't divide by 0")
4    }
5
6    return a / b, nil
7  }
8
9  func main() {
10   result, err := divide(10, 2)
11   if err != nil {
12     fmt.Println("Error caught:", err)
13     return
14   }
15
16   fmt.Println("Success, result was: ", result)
17 }
```

Go Errors

- Return type is `(int, error)`
- Return `nil` error if success
- Check both when function is called

```
2    if b == 0 {
3        return 0, errors.New("Can't divide by 0")
4    }
5
6    return a / b, nil
7  }
8
9  func main() {
10   result, err := divide(10, 2)
11   if err != nil {
12     fmt.Println("Error caught:", err)
13     return
14   }
15
16   fmt.Println("Success, result was: ", result)
17 }
```

Go Errors

- Return type is `(int, error)`
- Return `nil` error if success
- Check both when function is called

```
2    if b == 0 {
3        return 0, errors.New("Can't divide by 0")
4    }
5
6    return a / b, nil
7  }
8
9  func main() {
10   result, err := divide(10, 2)
11   if err != nil {
12     fmt.Println("Error caught:", err)
13     return
14   }
15
16   fmt.Println("Success, result was: ", result)
17 }
```

Go Errors

- Return type is `(int, error)`
- Return `nil` error if success
- Check both when function is called

```go
func divide(a, b int) (int, error) {
  if b == 0 {
      return 0, errors.New("Can't divide by 0")
  }

  return a / b, nil
}

func main() {
  result, err := divide(10, 2)
  if err != nil {
    fmt.Println("Error caught:", err)
    return
  }

  fmt.Println("Success, result was: ", result)
```

Go Errors

- Solves both problems at once
- Simple solution, but can get annoying
- Codebase is filled with `if err != nil {...}`
- Also must check *what* the error actually is
- Let's look at how Rust handles this

Rust Errors

- Rust errors are *enums*
- Come in two separate forms:
- `Result` -> Code that may/may not succeed
- `Option` -> Code that may/may not return a value
- We'll look at `Result` today, but they are both similar

Result

- Generic Type defined via standard library

```
1 enum Result<T, E> {
2     Ok(T), // success case
3     Err(E), // error case
4 }
```

- Return this from your functions instead
- Define what each generic is
- Error type is often defined via standard library
- But can be *any type* (e.g `String`)
- Let's see an example:

Result Usage

- Return a `Result<f64, String>`
- `Err(...)` creates a Result<_, Err>
- `Ok(...)` creates a Result<Data, _>

```
fn divide(num: f64, denom: f64) -> Result<f64, String> {
    if denom == 0.0 {
        Err(String::from("Cannot divide by zero"))
    } else {
        Ok(num / denom)
    }
}
```

Handling Results

- Pattern `match` and unpack the output
- Use the data within scope of the match
- Progress, but some issues:

```
1  fn main() {
2      let x = 10.0;
3      let y = 0.0;
4
5      match divide(x, y) {
6          Ok(value) => println!("Success! Result: {value}"),
7          Err(error) => println!("Error occurred: {error}"),
8      }
9
10 }
```

Extracting data

- `value` is only valid within the match
- Probably want to use the result later in the code
- What should `res` be in the `Err(error)` case

```
let res = match divide(x, y) {
    Ok(value) => value,
    Err(error) => ???,
};
```

Data Extraction

- Eventually, there will be a sensible value to return
- "If this goes wrong, use this instead"
- Sometimes the option might be "stop the program!"
- We call this *panicking*
- Use `.unwrap()` if we want the data now or panic

```
1 fn main() {
2     let x = 10.0;
3     let y = 0.0;
4
5     let res = divide(x, y).unwrap();
6 }
```

Don't Panic

- Panicking is not *idiomatic*, avoid it at all costs
- Very often, if a function you call can go wrong
- Your function can also go wrong
- Rust provides a mechanism to *propagate* your errors
- The ? operator unpacks a value, or returns the *same* error you just received

Propagation

- ? sends error to the caller
- Otherwise, unpacks the value

```
// use the () type for nothing
fn run_division() -> Result<(), String> {
    // send an error to the caller, or unpack the value
    let result = divide(10.0, 2.0)?;

    println!("Division was: {result}");

    Ok(())
}
```

- Let's see full example

Propagation In Action

```rust
fn divide(num: f64, denom: f64) -> Result<f64, String> {
    if denom == 0.0 {
        Err(String::from("Cannot divide by zero"))
    } else {
        Ok(num / denom)
    }
}

// use the () type for nothing
fn run_division() -> Result<(), String> {
    // send an error to the caller, or unpack the value
    let result = divide(10.0, 2.0)?;

    println!("Division was: {result}");

    Ok(())
```

Propagation In Action

```rust
fn divide(num: f64, denom: f64) -> Result<f64, String> {
    if denom == 0.0 {
        Err(String::from("Cannot divide by zero"))
    } else {
        Ok(num / denom)
    }
}

// use the () type for nothing
fn run_division() -> Result<(), String> {
    // send an error to the caller, or unpack the value
    let result = divide(10.0, 2.0)?;

    println!("Division was: {result}");

    Ok(())
```

Propagation In Action

```
 6     }
 7 }
 8
 9 // use the () type for nothing
10 fn run_division() -> Result<(), String> {
11     // send an error to the caller, or unpack the value
12     let result = divide(10.0, 2.0)?;
13
14     println!("Division was: {result}");
15
16     Ok(())
17 }
18
19 // you can actually have main return a Result
20 fn main() -> Result<(), String> {
21
```

Propagation In Action

```rust
fn run_division() -> Result<(), String> {
    // send an error to the caller, or unpack the value
    let result = divide(10.0, 2.0)?;

    println!("Division was: {result}");

    Ok(())
}

// you can actually have main return a Result
fn main() -> Result<(), String> {

    let res = run_division()?;

    Ok(())
}
```

Propagation In Action

```rust
fn divide(num: f64, denom: f64) -> Result<f64, String> {
    if denom == 0.0 {
        Err(String::from("Cannot divide by zero"))
    } else {
        Ok(num / denom)
    }
}

// use the () type for nothing
fn run_division() -> Result<(), String> {
    // send an error to the caller, or unpack the value
    let result = divide(10.0, 2.0)?;

    println!("Division was: {result}");

    Ok(())
```

Summary

- C-style error handling had `NULL` everywhere
- Java/Python-style exceptions contain hidden gotos
- Error's as value languages treat errors as regular data
- Rust does this with the `Result` enum
- We will continue error handling next time

Questions?